AIR CANVAS – FINAL REPORT

Air Canvas: Real-Time Virtual Drawing Using Hand Motion

Submitted by:

Anushka Gupta
Roll No: 102303358
ELC Activity: Real-Time Object Detection and Drawing Effects
Academic Year: 2024–2025
Institution: Thapar Institute of Engineering & Technology

Introduction

In the evolving world of computer vision and interactive systems, touchless interfaces are becoming more relevant than ever. The Air Canvas project enables users to draw in the air using a webcam and a colored object (e.g., a pen cap), without requiring any physical contact with the screen. Using Python and OpenCV, this project tracks the motion of a colour-detected object and draws on a virtual canvas in real-time. The system acts as a basic prototype of gesture-based drawing systems used in AR/VR and educational tools.

Objective

The primary objective of the Air Canvas project is to build an intuitive, real-time drawing application that allows users to draw in the air using only a webcam and a colored object as a pointer. This system removes the need for touchscreens or physical drawing devices by using computer vision techniques for gesture-based interaction. The system should support:

- Drawing continuous strokes on a virtual canvas
- Colour selection via on-screen regions
- Canvas clearing functionality
- Live colour tracking via HSV (Hue-Saturation-Value) tuning

In essence, the aim is to prototype an interactive, contactless drawing experience using Python and OpenCV.

Tools and Technologies Used

The project utilises a minimal yet powerful technology stack to achieve real-time performance and flexibility.

- **Programming Language:**
  Python 3.x was used for its simplicity, strong OpenCV support, and quick prototyping ability.

- **Computer Vision Library:**
  OpenCV (Open Source Computer Vision Library) handles image acquisition, colour space conversion, masking, contour detection, and drawing logic.

- **Numerical Computing Library:**
  NumPy is used for matrix operations and defining kernel filters used in the mask creation process.

- **Hardware Requirements:**
  - Standard laptop with webcam
  - No external hardware is needed

- **Development Tools:**
  - Visual Studio Code for coding
  - Command Prompt/Terminal for execution
  - System microphone and speakers are not required

Working Methodology (Step-by-Step)

The Air Canvas project works by converting webcam video into colour-detectable HSV format and identifying a specific colour object (pen cap, etc.) in real-time. Here's how the system functions:

Step 1: Video Capture

The system starts by accessing the default webcam and continuously reading frames.

```
cap = cv2.VideoCapture(0)

ret, frame = cap.read()

frame = cv2.flip(frame, 1)
```

Step 2: Colour Space Conversion

Each frame is converted from BGR (Blue-Green-Red) to HSV (Hue-Saturation-Value) format. HSV is more stable under lighting variations and easier for colour detection.

```
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
```

Step 3: HSV Range Tuning via Trackbars

The user is provided with six trackbars (Upper Hue, Upper Saturation, Upper Value, Lower Hue, Lower Saturation, Lower Value) to define the colour range of the object they're using. This helps isolate only the object's colour from the rest of the image.

```
lower_hsv = np.array([l_hue, l_saturation, l_value])

upper_hsv = np.array([u_hue, u_saturation, u_value])

mask = cv2.inRange(hsv, lower_hsv, upper_hsv)
```

Step 4: Mask Creation and Contour Detection

A binary mask is created where only the pixels within the defined HSV range are white. Morphological operations like erosion, opening, and dilation help reduce noise. Contours are then detected in this mask to locate the pointer.

Object detection:

```
contours, _ = cv2.findContours(mask.copy(), cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
```

Contours are extracted from the mask to locate the colored object. The largest contour is tracked.

Step 5: Pointer Tracking and Canvas Interaction

The centre of the largest contour is used as the pointer's current location. If the pointer is near the top bar (y < 65), it's interpreted as an interaction with on-screen buttons (e.g., switching colour or clearing the canvas). Otherwise, it draws a line segment from the previous pointer location to the new one in the selected colour.

Drawing on Canvas

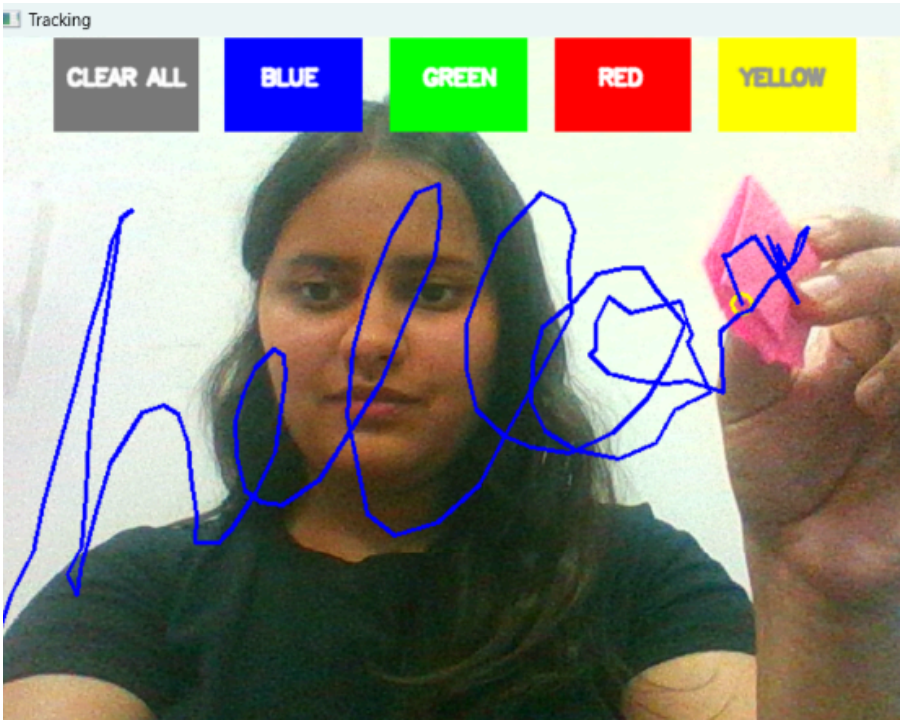
cv2.line(frame, prev_point, current_point, selected_color, 2)

Step 6: Real-Time Drawing

Lines are drawn both on the live video feed and a separate canvas. Multiple colours can be used, and users can clear the canvas at any time by hovering over the "CLEAR" button.

Step 7: Exit

The program ends when the user presses the q key, releasing the webcam and closing all windows.



HSV Range adjusted for Pink (Magenta/Fuchsia tone):

Trackbar Name	Suggested Value
Lower Hue	145
Lower Saturation	100
Lower Value	100
Upper Hue	170
Upper Saturation	255
Upper Value	255

Key Features

Virtual Drawing

Users can draw on a virtual canvas by simply moving a colored object (like a pen cap) in front of the webcam. No physical interaction is needed.

Color Selection

The application has colour buttons (Blue, Green, Red, Yellow) displayed on the screen. Users can select the drawing colour by hovering their object over these buttons.

Real-Time HSV Tuning

Six HSV trackbars allow the user to tune the detection range dynamically. This

improves accuracy across different lighting conditions or object colors.

Canvas Clearing
A "CLEAR" button on the screen allows the user to reset the canvas instantly when hovered over.

Efficient Design
The project uses lightweight libraries (OpenCV, NumPy) and runs on standard hardware without requiring any additional tools or installations.

Applications

Touchless User Interfaces
This technology can be integrated into public displays, ATMs, or kiosks where physical contact should be minimised.

Education & Learning
An interactive tool for young learners to engage with digital content in classrooms or at home.

AR/VR Gesture Prototypes
This project is a base model for more advanced gesture-controlled systems used in augmented and virtual reality.

Accessibility Enhancement
For individuals with limited hand mobility, this approach can be adapted to create assistive input methods.

Creative Art Tools
It could be extended into a digital drawing app with brush styles, undo options, and save functionality.

Evaluation

The Air Canvas project was evaluated on performance, ease of use, and detection accuracy. Multiple colored objects were tested (pink cap, blue marker, neon green pen, with pink being the final colour) under varying lighting conditions.

Performance Results:

Parameter	Result (Out of 10)
Detection Accuracy	8.5
Real-time Responsiveness	9.0
HSV Tuning Effectiveness	8.0
System Robustness	7.5
Overall Usability	8.0

Testing Scenarios:

- **Bright indoor light:** Excellent detection
- **Natural daylight:** Slight overexposure, manageable
- **Dim lighting:** Reduced clarity, but still usable after tuning
- **Background distractions:** Occasionally triggered false positives if the background matched the object's colour

Overall, the application demonstrated reliable results and met its functional requirements effectively.

Limitations and Future Scope

Limitations:

- HSV detection is highly dependent on lighting conditions.
- Users must manually tune HSV values to calibrate for different objects.
- Cannot distinguish between multiple similar objects.
- Not ideal for environments with rapidly changing lighting.

Future Scope:

- Integration of deep learning-based object tracking (e.g., MediaPipe or YOLO)
- Add gesture recognition for actions like "undo", "save", or "exit"
- Allow canvas saving and export as image or PDF
- Implement voice controls for hands-free mode selection
- Make the system compatible with touchless interactive boards or smart TVs

Conclusion

The Air Canvas project effectively demonstrates a real-time, contactless drawing system using computer vision. Built using Python and OpenCV, it allows users to interact with a digital canvas through motion and color tracking, without requiring any hardware apart from a basic webcam. The project serves as a practical introduction to computer vision and can be extended for real-world touchless applications. It is a valuable prototype in the space of gesture-based interaction and accessible interfaces.